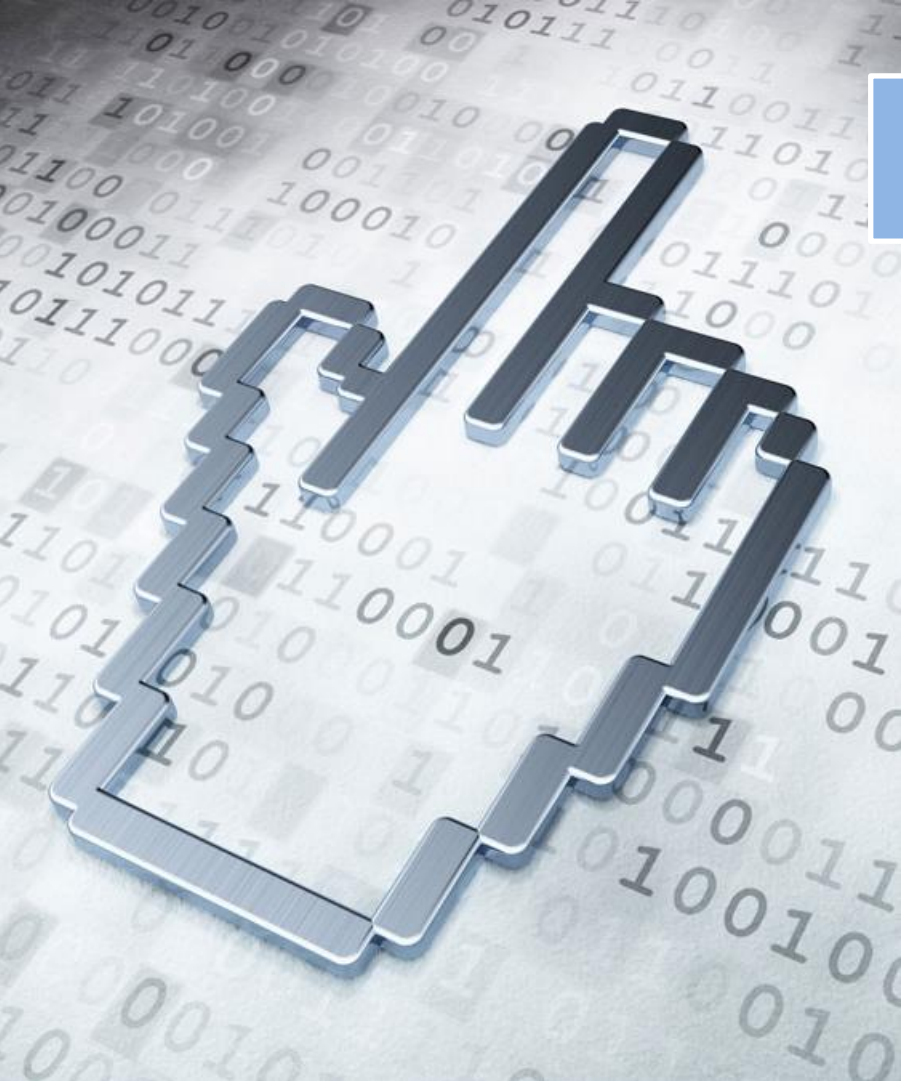




提纲

4.1 自顶向下的分析

4.2 预测分析法

4.3 自底向上的分析

4.4 LR分析法

4.5 语法分析器自动生成工具

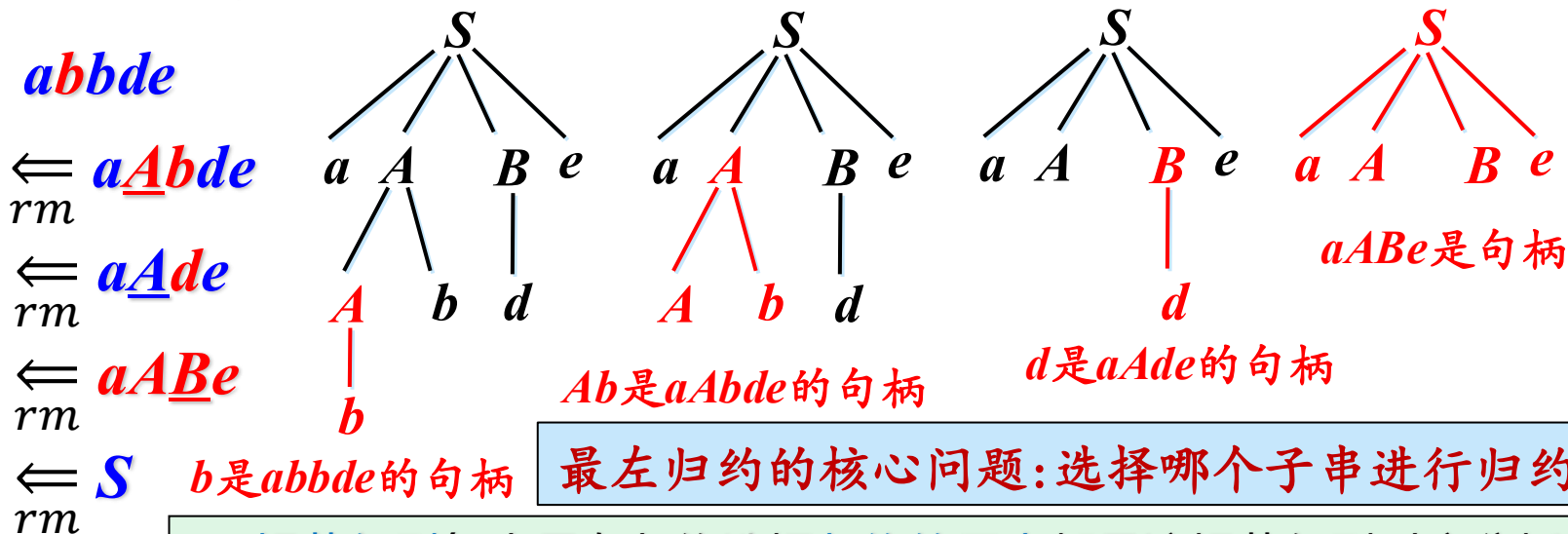
4.3 自底向上的语法分析

- 任务：从分析树的底部(叶节点)向顶部(根节点)方向构造分析树，可以看成是将输入串 w 归约为文法开始符号 S 的过程。
- 核心思想：从左到右扫描输入串，反复选择产生式进行最左归约（反向构建最右推导），如果最后能归约出文法的开始符号，则输入串是合法的句子，否则有语法错误。

目标：构建最左归约过程

➤ 设文法G为： $S \rightarrow aABe$ $A \rightarrow Ab|b$ $B \rightarrow d$

句子分析： *abbde*



➤ **规范句型** 每步最左归约选择 **归约的子串** 都是该规范句型对应分析树的最左直接短语(只有父子两代子树的边缘)，称为 **规范句型** 的 **句柄**。

句柄 (*handle*)

- 最左归约每步得到的都是规范句型（最右句型）

$S^* \xRightarrow{rm} \alpha A w \xRightarrow{rm} \alpha \beta w$, 称 β 是规范句型 $\alpha \beta w$ 的句柄

- 从句子开始，每步都选择句柄进行归约得到的总是规范句型
- 规范句型句柄右侧的串 w 一定是只包含终结符号。
- 句柄一定是某个产生式的右部，反之不成立。

如何在规范句型中找到句柄呢？

自底向上分析的通用框架——移入-归约分析

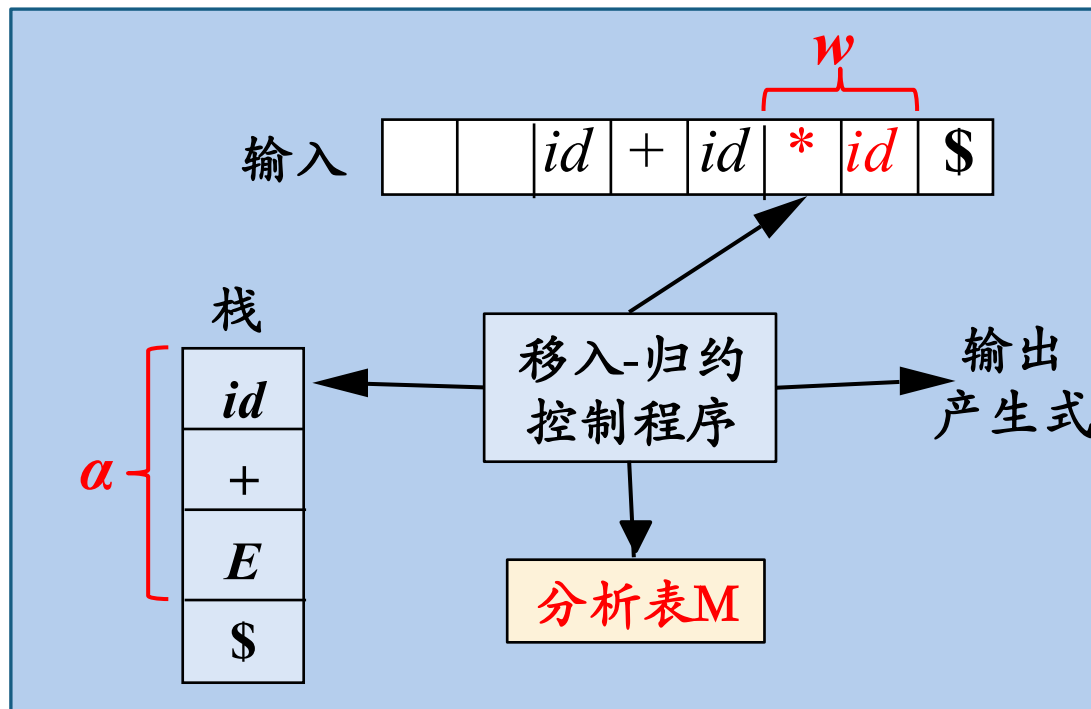
➤ 分析栈

保存处于分析中的文法符号串

➤ 初始： 栈：\$
输入：w\$

➤ 分析过程

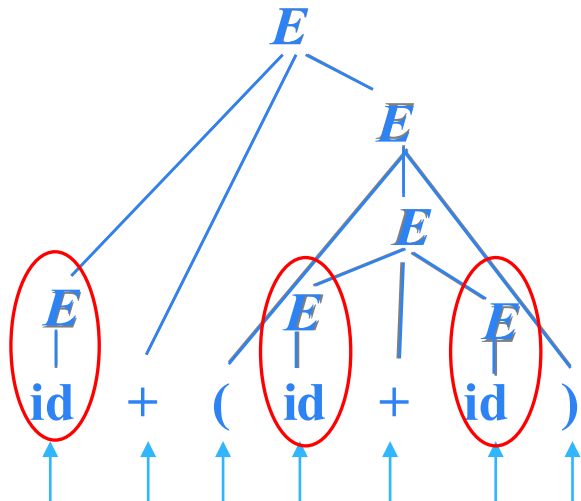
从左到右扫描输入串，将输入符号移入栈，一旦句柄在栈顶形成，就将其归约为对应的产生式头。重复这个过程，直到分析结束。



例：移入-归约分析

文法

- ① $E \rightarrow E+E$
- ② $E \rightarrow (E)$
- ③ $E \rightarrow id$



最左归约

栈

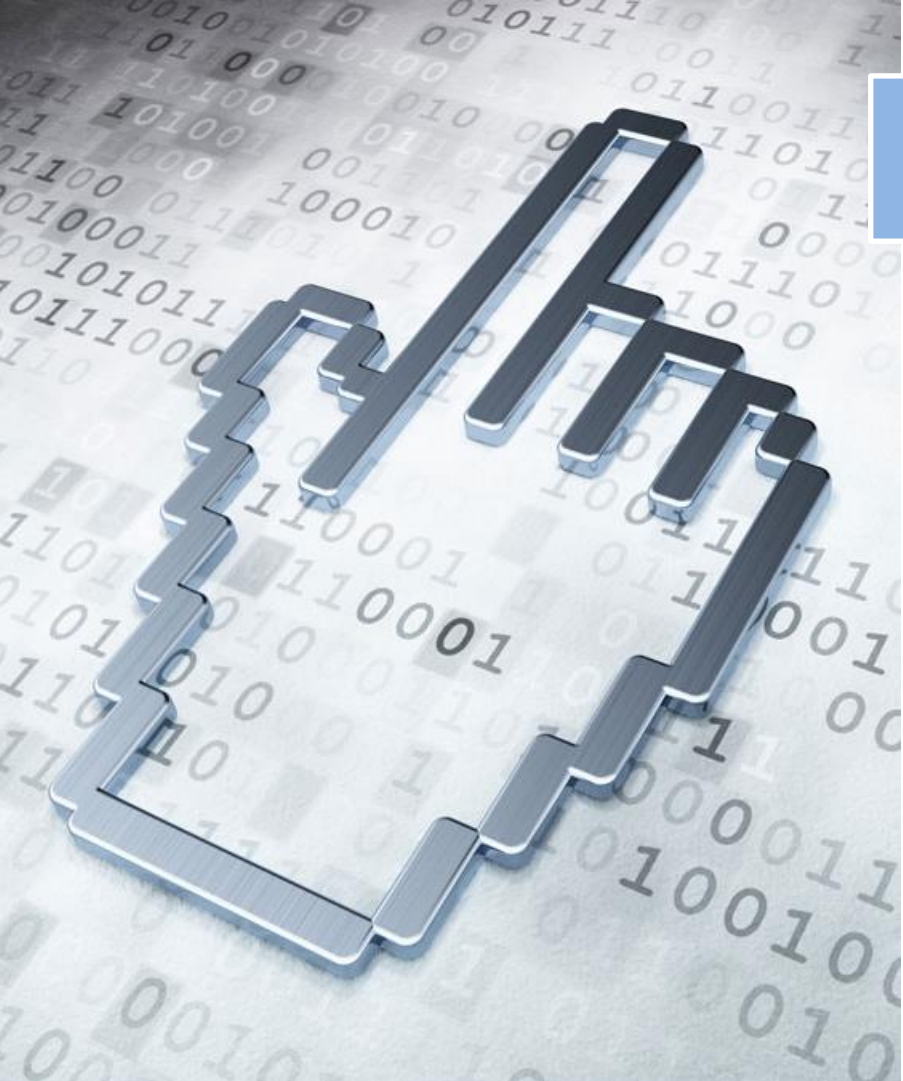
\$
\$ id
\$ E
\$ E+
\$ E+(
\$ E+(id
\$ E+(E
\$ E+(E+
\$ E+(E+id
\$ E+(E+E
\$ E+(E
\$ E+(E)
\$ E+E
\$ E

剩余输入

id+(id+id) \$
+(id+id) \$
+(id+id) \$
(id+id) \$
id+id) \$
+id) \$
+id) \$
id) \$
) \$
) \$
) \$
\$
\$
\$
\$

动作

移入
归约: $E \rightarrow id$
移入
移入
移入
归约: $E \rightarrow id$
移入
移入
归约: $E \rightarrow id$
归约: $E \rightarrow E+E$
移入
归约: $E \rightarrow (E)$
归约: $E \rightarrow E+E_6$



提纲

4.1 自顶向下的分析

4.2 预测分析法

4.3 自底向上的分析

4.4 LR分析法

4.5 语法分析器自动生成工具

4.4 LR 分析法 (状态法)

- LR分析技术是美国计算机科学家高德纳 (Donald Knuth) 于1965年首先提出来的。
- 优点：适用范围广；分析速度快；报错准确。
 - LR分析方法是已知的最通用的无回溯的移进—归约分析方法；
 - 几乎所有描述程序设计语言的上下文无关文法，都能够构造LR语法分析器；
 - 可以使用LR分析法的文法类是可以使用LL分析法的文法类的真超集。

4.4 LR 分析法

- $LR(k)$ 分析法可分析 $LR(k)$ 文法产生的语言
 - L ：从左到右扫描输入符号
 - R ：反向构造最右推导（最左归约）
 - k ：超前读入 k 个符号，以便确定分析器的动作。
 - $k=0$ 和 $k=1$ 这两种情况具有实践意义
 - 当省略 (k) 时，表示 $k=1$

移入-归约分析栈中的内容有什么特点?

- **活前缀 (可行前缀)** : 不包含句柄右侧符号的规范句型的前缀
- 规范句型所有活前缀构成正则语言
- **DFA+栈**来识别和管理活前缀

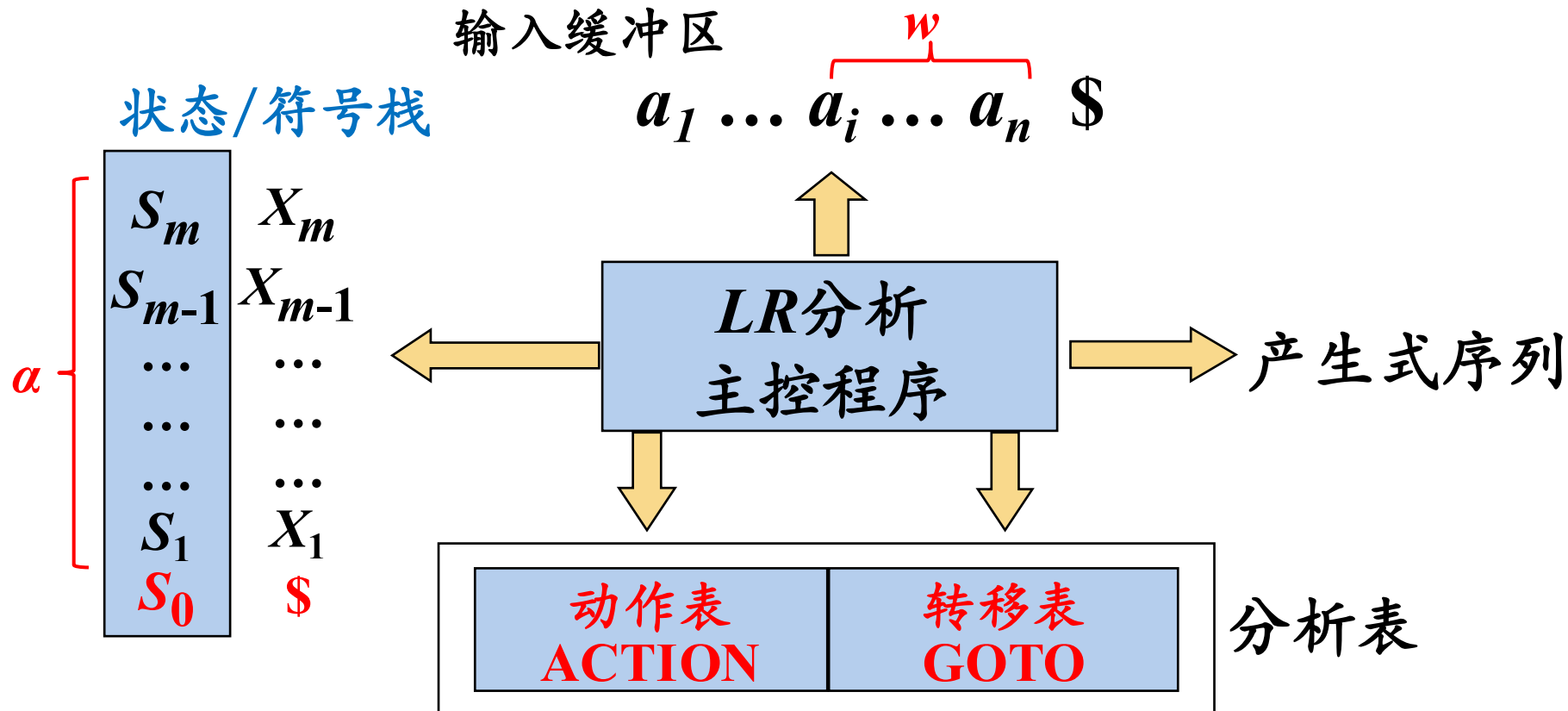
文法

- ① $E \rightarrow E+E$
- ② $E \rightarrow (E)$
- ③ $E \rightarrow \text{id}$

对句柄的识别就转变成对规范句型活前缀的识别

栈	剩余输入	动作
\$	id+(id+id) \$	
\$ id	+(id+id) \$	移入
\$ E	+(id+id) \$	归约: $E \rightarrow \text{id}$
\$ E+	(id+id) \$	移入
\$ E+(	id+id) \$	移入
\$ E+(id	+id) \$	移入
\$ E+(E	+id) \$	归约: $E \rightarrow \text{id}$
\$ E+(E+	id) \$	移入
\$ E+(E+id	) \$	移入
\$ E+(E+E	) \$	归约: $E \rightarrow \text{id}$
\$ E+(E	) \$	归约: $E \rightarrow E+E$
\$ E+(E)	\$	移入
\$ E+E	\$	归约: $E \rightarrow (E)$
\$ E	\$	归约: $E \rightarrow E+E$

LR 分析器的总体结构



LR 分析表的结构

➤ 例：文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$

➤ 不同的LR(k)分析法，表项的构建方法不同，但结构相同，分析过程相同

动作表

转移表

状态	ACTION			GOTO	
	<i>a</i>	<i>b</i>	\$	<i>S</i>	<i>B</i>
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

sn : 移入动作(shift action), 将状态*n*(代表当前输入符号) 移入栈
rn : 归约动作(reduce action), 用第*n*个产生式进行归约

LR 分析表的结构

➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$

状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

B
 $|$
 输入 $b \ a \ b$

栈 剩余输入
 0 4
 $\$ B$ $b \ a \ b \ \$$

LR 分析表的结构

➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$

输入 $\begin{array}{cc} B & B \\ | & | \\ b & a & b \end{array}$

状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈 剩余输入

0 2 3 4

$\$ B$

$ab \$$

LR 分析表的结构

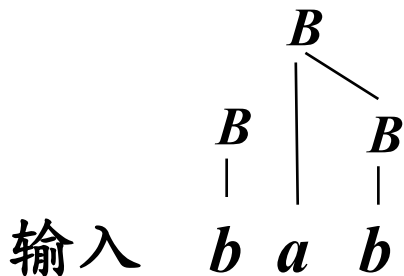
➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$



状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈

剩余输入

0 2 3 6

$\$ B a B$

$\$$

LR 分析表的结构

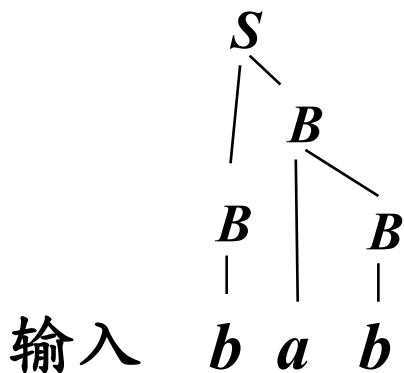
➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$



状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈

剩余输入

0 2 5

$\$ B B$

$\$$

LR 分析表的结构

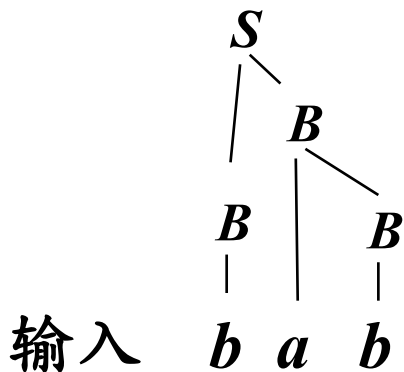
➤ 例

➤ 文法

① $S \rightarrow BB$

② $B \rightarrow aB$

③ $B \rightarrow b$



状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

栈

剩余输入

0 1

$\$ S$

$\$$

分析成功!

LR 分析器的工作过程

➤ 初始化

s_0
$\$ \qquad \qquad \qquad a_1 a_2 \dots a_n \$$

➤ 一般情况下

$s_0 s_1 \dots s_m$
$\$ X_1 \dots X_m \qquad \qquad a_i a_{i+1} \dots a_n \$$

① 如果 **ACTION** $[s_m, a_i] = sx$, 那么格局变为:

$s_0 s_1 \dots s_m x$
$\$ X_1 \dots X_m a_i \qquad \qquad a_{i+1} \dots a_n \$$

LR 分析器的工作过程

➤ 初始化

s_0	
\$	$a_1 a_2 \dots a_n$ \$

➤ 一般情况下

$s_0 s_1 \dots s_m$	
$\$X_1 \dots X_m$	$a_i a_{i+1} \dots a_n$ \$

② 如果 **ACTION** $[s_m, a_i] = rx$ 表示用第 x 个产生式 $A \rightarrow X_{m-(k-1)} \dots X_m$

进行归约，设 $|X_{m-(k-1)} \dots X_m| = k$ ，如果 **GOTO** $[s_{m-k}, A] = y$ ，那么格局变为：

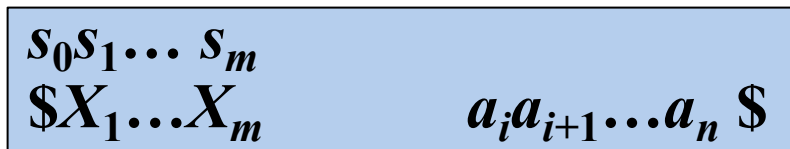
$s_0 s_1 \dots s_{m-k}$	y	
$\$X_1 \dots X_{m-k}$	A	$a_i a_{i+1} \dots a_n$ \$

LR 分析器的工作过程

➤ 初始化



➤ 一般情况下



③ 如果 $\text{ACTION}[s_m, a_i] = acc$ ，那么分析成功

④ 如果 $\text{ACTION}[s_m, a_i] = err$ ，那么出现语法错误

LR 分析算法

- 输入：串 w 和LR语法分析表，该表描述了文法 G 的ACTION函数和GOTO函数。
- 输出：如果 w 在 $L(G)$ 中，则输出 w 的自底向上语法分析过程中的归约步骤；否则给出一个错误指示。
- 方法：初始时，语法分析器栈中的内容为初始状态 s_0 ，输入缓冲区中的内容为 $w\$$ 。然后，语法分析器执行下面的程序：

```
令 $a$ 为 $w\$$ 的第一个符号；
while(1) { /* 永远重复*/
    令 $s$ 是栈顶的状态；
    if ( ACTION [ $s, a$ ] = 移入 $t$  ) {
        将状态 $t$ 压入栈中； // 不需要压入符号
        令 $a$ 为下一个输入符号；
    } else if ( ACTION [ $s, a$ ] = 归约 $A \rightarrow \beta$  ) {
        从栈中弹出  $|\beta|$  个状态（符号）；
        将GOTO [ $t, A$ ]压入栈中；
        输出产生式 $A \rightarrow \beta$ ；
    } else if ( ACTION [ $s, a$ ] = 接受 ) break; /* 语法分析完成*/
```

如何构造给定文法的 LR 分析表?

➤ $LR(0)$ 分析

➤ SLR 分析

➤ $LR(1)$ 分析

➤ $LALR$ 分析

4.4.1 LR(0) 分析

- 句柄是逐步形成的，用“项目”表示句柄识别的进展状态
- **项目**：右部某位置标有圆点的产生式称为相应文法的一个**LR(0)项目**（简称为项目）

$$A \rightarrow \alpha_1 \cdot \alpha_2$$

例： $S \rightarrow bBB$

➤ $S \rightarrow \cdot bBB$ ← 移入项目

➤ $S \rightarrow b \cdot BB$

➤ $S \rightarrow bB \cdot B$

➤ $S \rightarrow bBB \cdot$ ← 归约项目

} 待约项目

产生式 $A \rightarrow \varepsilon$ 只生成一个项目 $A \rightarrow \cdot$

增广文法 (*Augmented Grammar*)

➤ 如果 G 是一个以 S 为开始符号的文法，则 G 的**增广文法** G' 就是在 G 中加上新开始符号 S' 和产生式 $S' \rightarrow S$ 而得到的文法

➤ 例

1) $E \rightarrow E + T$
2) $E \rightarrow T$
3) $T \rightarrow T * F$
4) $T \rightarrow F$
5) $F \rightarrow (E)$
6) $F \rightarrow \text{id}$



0) $E' \rightarrow E$
1) $E \rightarrow E + T$
2) $E \rightarrow T$
3) $T \rightarrow T * F$
4) $T \rightarrow F$
5) $F \rightarrow (E)$
6) $F \rightarrow \text{id}$

引入这个新的开始产生式的目的是使得文法开始符号仅出现在一个产生式的左边，从而使得分析器只有一个接受状态

文法中的LR(0)项目

① $S' \rightarrow S$ ② $S \rightarrow vI:T$ ③ $I \rightarrow I,i$ ④ $I \rightarrow i$ ⑤ $T \rightarrow r$

(2) $S \rightarrow \cdot vI:T$

初始项目

(3) $S \rightarrow v \cdot I:T$

(7) $I \rightarrow \cdot I,i$

(4) $S \rightarrow vI \cdot :T$

(8) $I \rightarrow I \cdot ,i$

(0) $S' \rightarrow \cdot S$

(5) $S \rightarrow vI: \cdot T$

(9) $I \rightarrow I, \cdot i$

(11) $I \rightarrow \cdot i$

(13) $T \rightarrow \cdot r$

(1) $S' \rightarrow S \cdot$

(6) $S \rightarrow vI:T \cdot$

(10) $I \rightarrow I,i \cdot$

(12) $I \rightarrow i \cdot$

(14) $T \rightarrow r \cdot$

接收项目

归约项目

- 移进项目：形如 $A \rightarrow \alpha \cdot a\beta$ ，其中 $a \in V_T$ 分析时，需要把 a 移进栈
- 待约项目：形如 $A \rightarrow \alpha \cdot B\beta$ ，其中 $B \in V_N$ ，等待归约出 B
- 归约项目：形如 $A \rightarrow \alpha \cdot$ 句柄已形成，可以归约

文法中的LR(0)项目

① $S' \rightarrow S$ ② $S \rightarrow vI:T$ ③ $I \rightarrow I,i$ ④ $I \rightarrow i$ ⑤ $T \rightarrow r$

(2) $S \rightarrow \cdot vI:T$

(3) $S \rightarrow v \cdot I:T$ (7) $I \rightarrow \cdot I,i$

(4) $S \rightarrow vI \cdot :T$ (8) $I \rightarrow I \cdot ,i$

(0) $S' \rightarrow \cdot S$ (5) $S \rightarrow vI: \cdot T$ (9) $I \rightarrow I, \cdot i$ (11) $I \rightarrow \cdot i$ (13) $T \rightarrow \cdot r$

(1) $S' \rightarrow S \cdot$ (6) $S \rightarrow vI:T \cdot$ (10) $I \rightarrow I,i \cdot$ (12) $I \rightarrow i \cdot$ (14) $T \rightarrow r \cdot$

➤ 后继项目 (Successive Item)

➤ 同属于一个产生式的项目，但圆点的位置只相差一个符号，则称后者是前者的后继项目

即： $A \rightarrow \alpha \cdot X\beta$ 的后继项目是 $A \rightarrow \alpha X \cdot \beta$

文法中的LR(0)项目

① $S' \rightarrow S$ ② $S \rightarrow vI:T$ ③ $I \rightarrow I,i$ ④ $I \rightarrow i$ ⑤ $T \rightarrow r$

(2) $S \rightarrow \cdot vI:T$

(3) $S \rightarrow v \cdot I:T$

(4) $S \rightarrow vI \cdot :T$

(7) $I \rightarrow \cdot I,i$

(8) $I \rightarrow I \cdot ,i$

(0) $S' \rightarrow \cdot S$

(5) $S \rightarrow vI \cdot :T$

(9) $I \rightarrow I \cdot ,i$

(11) $I \rightarrow \cdot i$

(13) $T \rightarrow \cdot r$

(1) $S' \rightarrow S \cdot$

(6) $S \rightarrow vI:T \cdot$

(10) $I \rightarrow I,i \cdot$

(12) $I \rightarrow i \cdot$

(14) $T \rightarrow r \cdot$

使用LR(0)项目可以构造LR(0)自动机，该自动机可以识别可能出现在栈中的所有串（活前缀），可用于做出语法分析决定。

例：LR(0)自动机

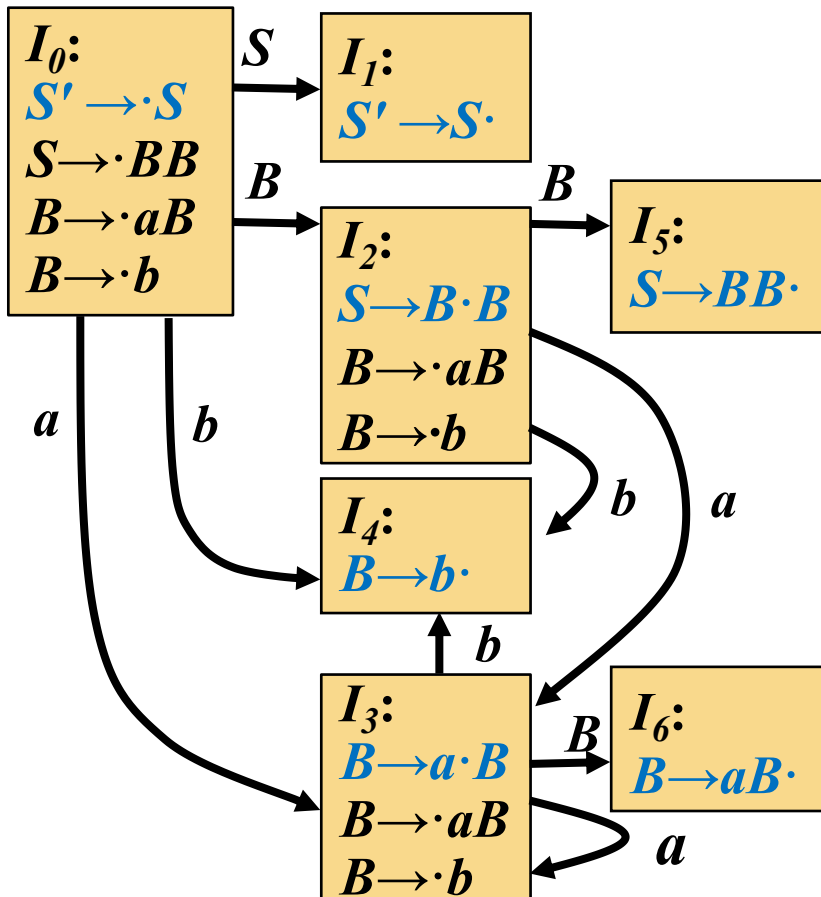
文法

(0) $S' \rightarrow S$

(1) $S \rightarrow BB$

(2) $B \rightarrow aB$

(3) $B \rightarrow b$



核心项目： $S' \rightarrow \cdot S$ 和所有不在最左端的项目。

状态：核心项目集合的闭包。

求项集闭包 $closure(I)$ ：
若 $A \rightarrow \alpha \cdot B \beta \in I$ ，则将 B 开头的产生式加入 I ，重复执行此规则，直到没有新产生式加入。

LR(0)项集规范族：
所有规范LR(0)项集的集合

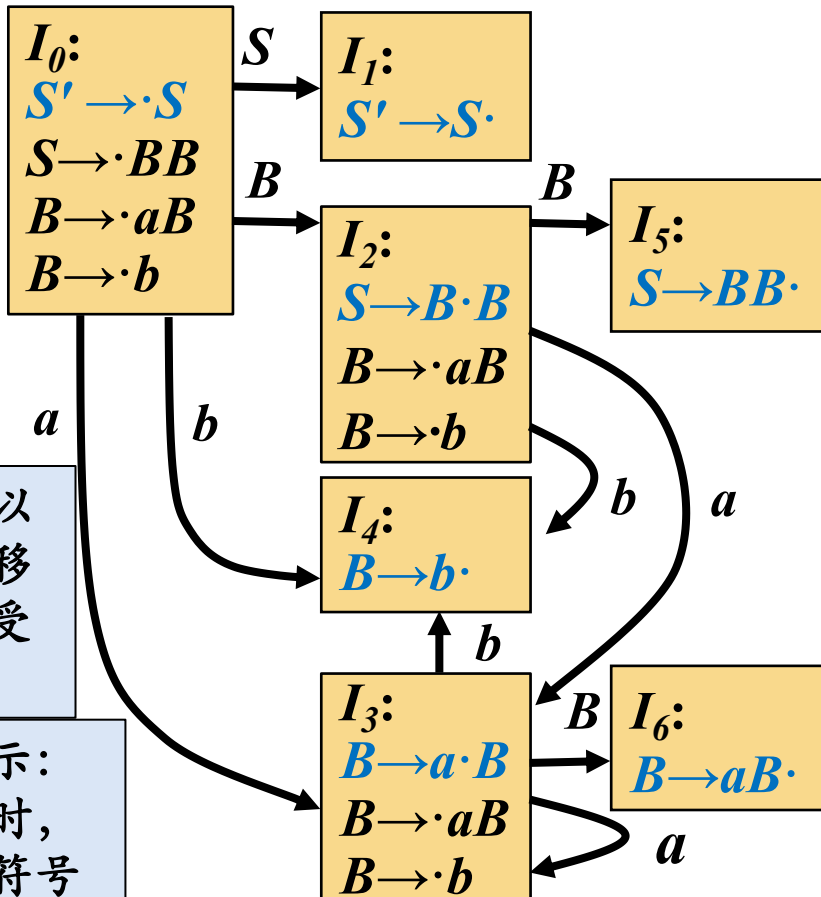
例：LR(0)自动机

文法

- (0) $S' \rightarrow S$
- (1) $S \rightarrow BB$
- (2) $B \rightarrow aB$
- (3) $B \rightarrow b$

状态中项目可以
指导栈操作（移
入、归约、接受
）

LR(0)中的0表示：
执行归约动作时，
不需查看输入符号



LR(0)分析表

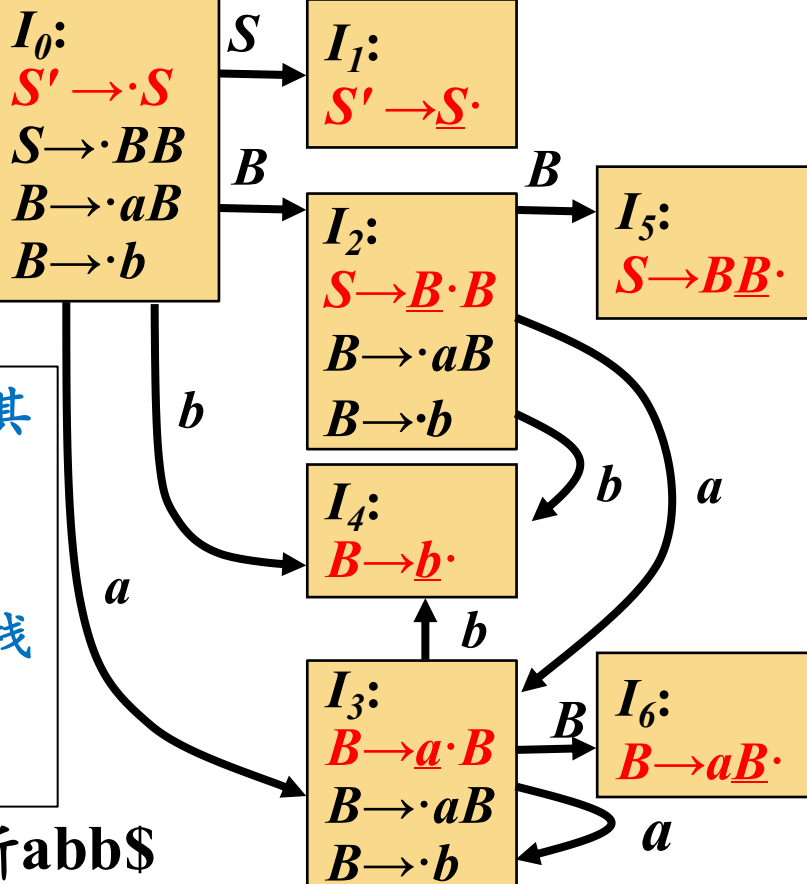
状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

例：LR(0)自动机

任一时刻分析栈中的符号串，一定是从**初始状态**到**栈顶状态**的**路径上对应的符号串**。

文法

- (0) $S' \rightarrow S$
- (1) $S \rightarrow BB$
- (2) $B \rightarrow aB$
- (3) $B \rightarrow b$



LR(0)分析表

状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

➤ 每个状态与其入边上标记的符号对应

➤ 状态代表了栈中所有符号的信息

例如：分析 $abb\$$

例：LR(0)自动机

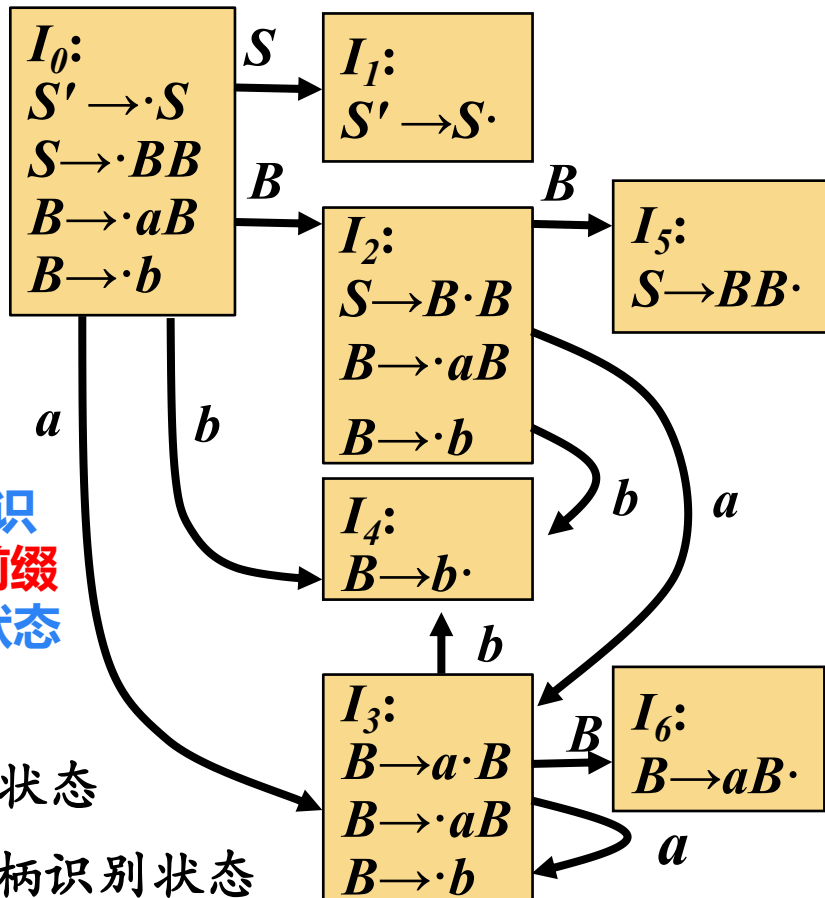
增广文法

- (0) $S' \rightarrow S$
- (1) $S \rightarrow BB$
- (2) $B \rightarrow aB$
- (3) $B \rightarrow b$

基于LR(0)项目识别文法全部活前缀的DFA。所有状态都是接收状态。

I_1 是句子识别状态

$I_1 I_4 I_5 I_6$ 是句柄识别状态



LR(0)分析表

状态	ACTION			GOTO	
	a	b	$\$$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

活前缀与句柄的关系

- 设 $A \rightarrow \beta$ 是句柄，栈中的规范句型活前缀与句柄的关系：
 - 不含句柄的任何符号： $A \rightarrow \cdot \beta$
 - 包含句柄的部分符号： $A \rightarrow \beta_1 \cdot \beta_2$
 - 包含句柄的全部符号： $A \rightarrow \beta \cdot$

构造LR分析表的算法

CLOSURE闭包函数和GOTO函数

➤ 计算项集 I 闭包: CLOSURE 闭包函数

$$\text{CLOSURE}(I) = I \cup \{ B \rightarrow \cdot \gamma \mid \exists A \rightarrow \alpha \cdot B \beta \in \text{CLOSURE}(I), B \rightarrow \gamma \in P \}$$

➤ 状态转移: GOTO 函数

$$\text{GOTO}(I, X) = \text{CLOSURE}(\{ A \rightarrow \alpha X \cdot \beta \mid \forall A \rightarrow \alpha \cdot X \beta \in I \})$$

➤ $A \rightarrow \alpha X \cdot \beta$ 称为 $A \rightarrow \alpha \cdot X \beta$ 的后继项目

➤ $\text{GOTO}(I, X)$ 称为项集 I 关于 X 的后继项目集。

CLOSURE()函数

```
SetOfItems CLOSURE (  $I$  ) {  
     $J = I$ ;  
    repeat  
        for (  $J$  中的每个项  $A \rightarrow \alpha \cdot B \beta$  )  
            for (  $G$  的每个产生式  $B \rightarrow \gamma$  )  
                if ( 项  $B \rightarrow \cdot \gamma$  不在  $J$  中 )  
                    将  $B \rightarrow \cdot \gamma$  加入  $J$  中;  
    until 在某一轮中没有新的项被加入到  $J$  中;  
    return  $J$ ;  
}
```

GOTO ()函数

➤ 返回项目集 I 对应于文法符号 X 的**后继**项目集闭包

$$\text{GOTO}(I, X) = \text{CLOSURE}(\{ A \rightarrow \alpha X \cdot \beta \mid \forall A \rightarrow \alpha \cdot X \beta \in I \})$$

```
SetOfItems GOTO (  $I$ ,  $X$  ) {  
    将 $J$ 初始化为空集;  
    for (  $I$  中的每个项  $A \rightarrow \alpha \cdot X \beta$  )  
        将项  $A \rightarrow \alpha X \cdot \beta$  加入到集合 $J$  中;  
    return CLOSURE (  $J$  );  
}
```

构造 $LR(0)$ 自动机的状态集

➤ 规范 $LR(0)$ 项集族(*Canonical $LR(0)$ Collection*)

$$C = \{ I_0 \} \cup \{ I \mid \exists J \in C, X \in V_N \cup V_T, I = GOTO(J, X) \}$$

```
void items(  $G'$  ) {  
     $C = \{ \text{CLOSURE} ( \{ [ S' \rightarrow \cdot S ] \} ) \};$   
    repeat  
        for ( $C$ 中的每个项集  $I$ )  
            for(每个文法符号  $X$ )  
                if (  $GOTO(I, X)$ 非空且不在 $C$ 中)  
                    将 $GOTO(I, X)$ 加入 $C$ 中;  
    until在某一轮中没有新的项集被加入到 $C$ 中;  
}
```

LR(0)分析表构造算法

- 构造 G' 的规范LR(0)项集族 $C = \{ I_0, I_1, \dots, I_n \}$
- 令 I_i 对应状态 i 。状态 i 的语法分析动作按照下面的方法决定：
 - if $A \rightarrow \alpha \cdot a \beta \in I_i, a \in V_T$ 且 $GOTO(I_i, a) = I_j$ then $ACTION[i, a] = sj$
 - if $A \rightarrow \alpha \cdot B \beta \in I_i, B \in V_N$ 且 $GOTO(I_i, B) = I_j$ then $GOTO[i, B] = j$
 - if $A \rightarrow \alpha \cdot \in I_i$ 且 $A \neq S'$ then for $\forall a \in V_T \cup \{\$ \}$ do $ACTION[i, a] = rj$ (j 是产生式 $A \rightarrow \alpha$ 的编号)
 - if $S' \rightarrow S \cdot \in I_i$ then $ACTION[i, \$] = acc$
- 没有定义的所有条目都设置为“error”

LR(0) 自动机的形式化定义

➤ 文法

$$G = (V_N, V_T, P, S)$$

➤ LR(0) 自动机

$$M = (C, V_N \cup V_T, GOTO, I_0, F)$$

$$➤ C = \{ I_0 \} \cup \{ I \mid \exists J \in C, X \in V_N \cup V_T, I = GOTO(J, X) \}$$

$$➤ I_0 = CLOSURE(\{ S' \rightarrow \cdot S \})$$

$$➤ F = C \text{ (活前缀的识别状态)}$$

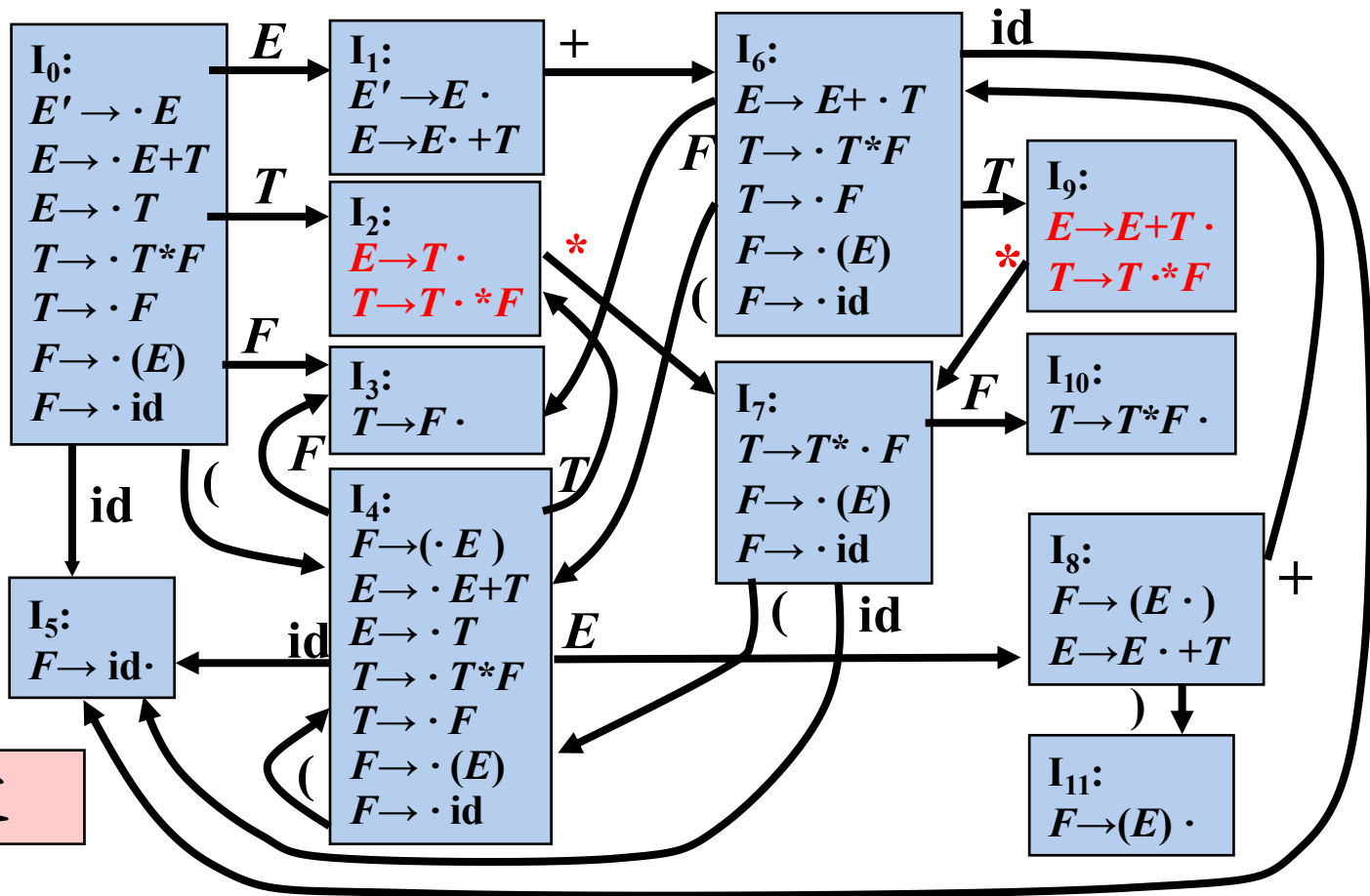
语法分析句子的识别状态：
 $F = \{ \text{包含项 } S' \rightarrow S \cdot \text{的状态} \}$

思考：哪些状态是句柄的识别状态？

LR(0) 分析过程中的冲突

文法

- (0) $E' \rightarrow E$
- (1) $E \rightarrow E+T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T*F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$



移入/归约冲突

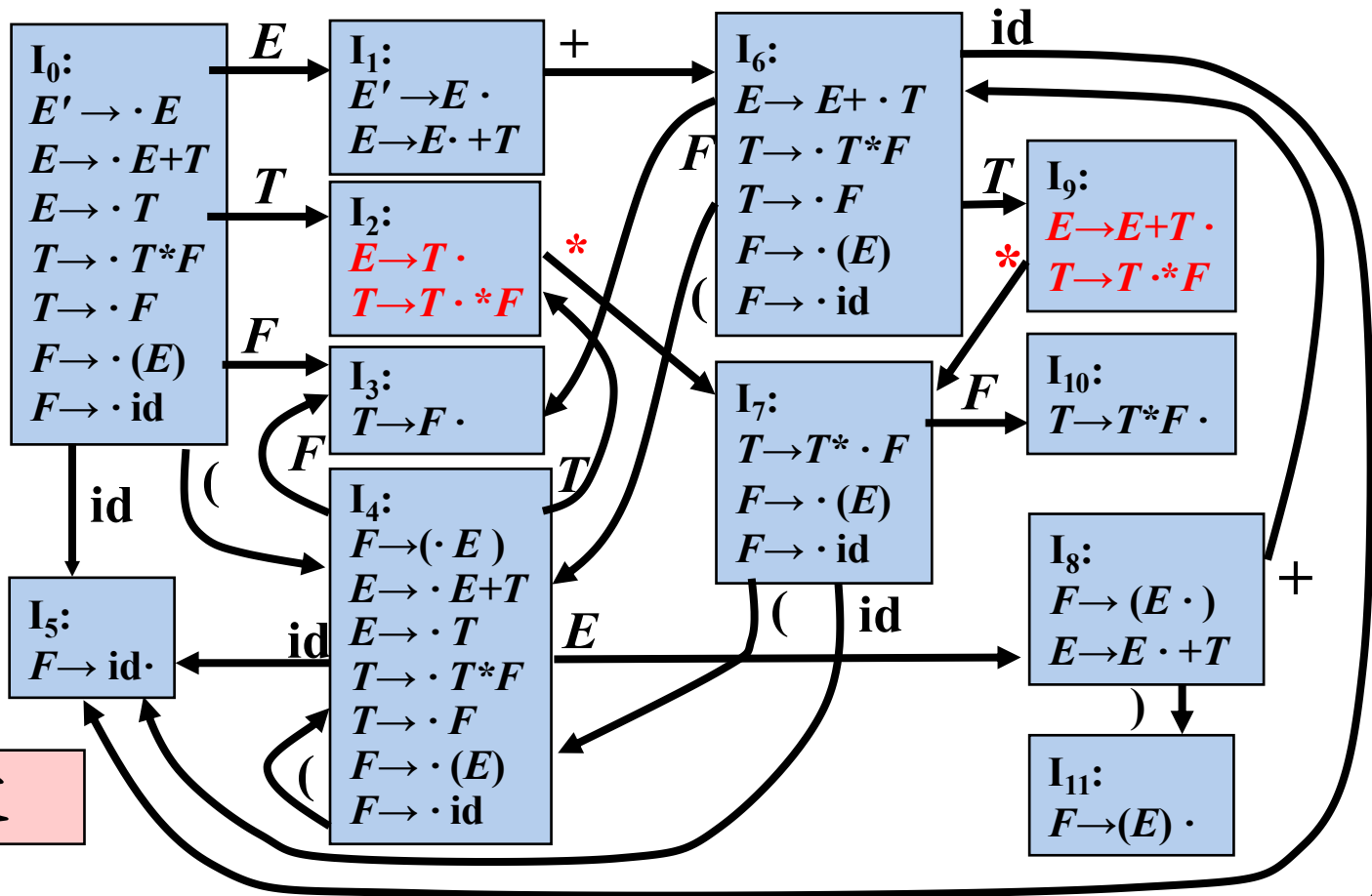
表达式文法的 $LR(0)$ 分析表含有移入/归约冲突

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2	r2	r2	r2/s7	r2	r2	r2			
3	r4	r4	r4	r4	r4	r4			
4	s5			s4			8	2	3
5	r6	r6	r6	r6	r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9	r1	r1	r1/s7	r1	r1	r1			
10	r3	r3	r3	r3	r3	r3			
11	r5	r5	r5	r5	r5	r5			

LR(0) 分析过程中的冲突

文法

- (0) $E' \rightarrow E$
- (1) $E \rightarrow E+T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T*F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow id$



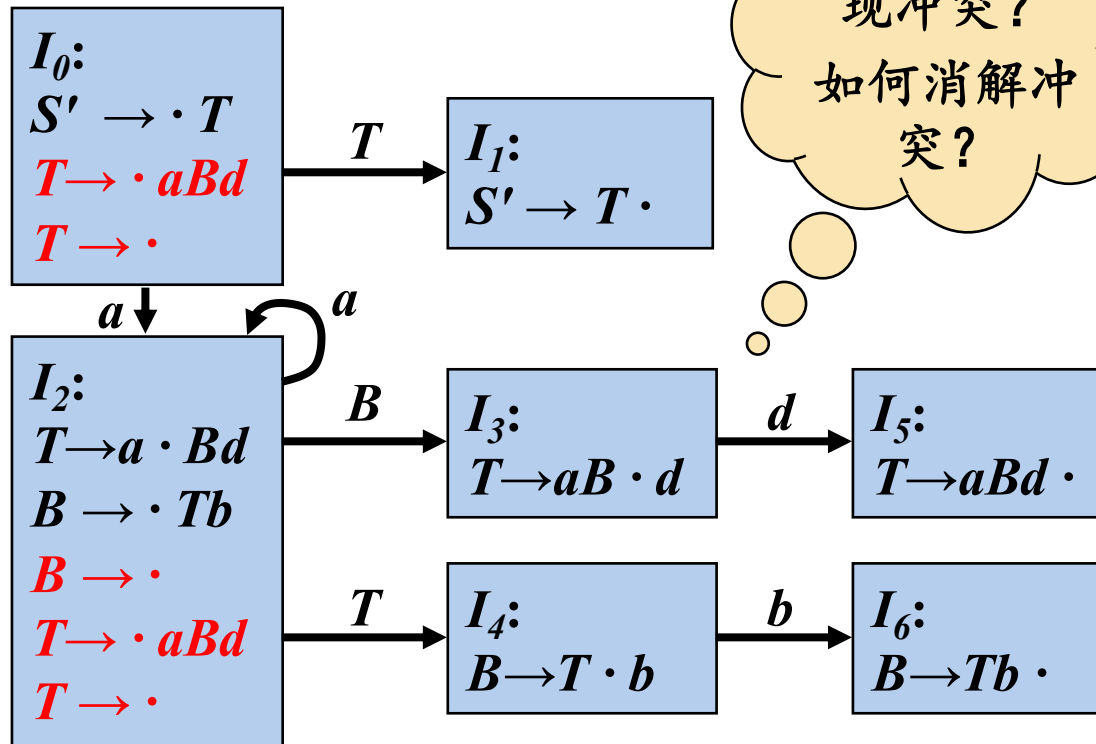
移入/归约冲突

例：移入/归约冲突和归约/归约冲突

文法

- (0) $S' \rightarrow T$
- (1) $T \rightarrow aBd$
- (2) $T \rightarrow \varepsilon$
- (3) $B \rightarrow Tb$
- (4) $B \rightarrow \varepsilon$

如果LR(0)分析表中
没有语法分析动作冲突，那么给定的文法
就称为LR(0)文法



不是所有CFG都能用LR(0)方法进行分析，也就是说，CFG不总是LR(0)文法

课后练习

- 画出文法的LR(0)自动机，并画出LR(0)分析表，说明是不是LR(0)文法，为什么？

$$S \rightarrow a \mid S + S \mid S S \mid S * \mid (S)$$

课程主要内容

1. 绪论 (2学时)
2. 语言及其文法 (2学时)
3. 词法分析 (3学时)
4. 语法分析 (9学时)
5. 语法制导翻译 (6学时)
6. 中间代码生成 (7学时)
7. 运行时的存贮组织 (3学时)
8. 代码优化 (6学时)
9. 代码生成 (2学时)